

RAPORT ARCHITEKTURY PROJEKTU ATHLETE PLATFORM

Data analizy: 29 lipca 2026

1. STRUKTURA PROJEKTU

Główne pakiety i ich odpowiedzialności:

- collectors/ : Pobieranie i import surowych danych (Apple Health XML, FIT , badania krwi, skład ciała, oddawanie krwi).
- core/ & schema/ : Znormalizowane modele domenowe (HealthDaily, TrainingDaily , BloodTest) oraz połączenie z bazą DuckDB (health.duckdb).
- repositories/ : Warstwa dostępu do danych z bazy DuckDB (HealthRepository, WorkoutRepository).
- engines/ & health/ : Analiza podstawowa: wyliczanie średnich kroczących (7d/30d) w ContextBuilder oraz wskaźników zdrowotnych.
- recovery/ & sleep/ : Analiza fizjologiczna regeneracji (obliczanie odchyleń H RV, tętna spoczynkowego RHR oraz ocena jakości snu).
- training/ & performance/ : Obliczanie metryk kolarskich/biegowych (NP, IF, TS S, FTP) oraz obciążenia długo- i krótkoterminowego (CTL/ATL/TSB).
- athlete/ : Agregacja wszystkich cząstkowych wyników analiz w spójny obiekt AthleteState.
- decision/ : Deterministyczny silnik podejmowania decyzji treningowych (Diagnosis, Prescription, Selection).
- planner/ : Generowanie treningów na podstawie receptur treningowych oraz kompilacja domenowego języka DSL.
- workout/ & timeline/ : Budowanie osi czasu treningu oraz eksport jednostek do formatu ZWO (Zwift/TrainerRoad).
- execution/ : Analiza wykonania treningu i ocena zgodności zrealizowanego pliku FIT z planem.
- coach/ & briefing/ & renderers/ : Prezentacja wyników, formatowanie porannego raportu i komunikacja z użytkownikiem/LLM.

Granice domenowe:

1. Domena Ingestii: Przekształcanie surowych źródeł na znormalizowane obiekty core/models.py.
2. Domena Analizy i Stanu: Przetwarzanie zdrowia/treningów i wystawianie obiektu AthleteState.
3. Domena Decyzyjna: Logika wyliczania receptury na podstawie AthleteState.
4. Domena Generowania Treningów: Transpilacja intencji decyzyjnej do wykonalnych bloków DSL.

2. PRZEPLYW DANYCH

1. Ingestia danych:

Kolektory (collectors/) pobierają dane z Apple Health / plików FIT / wyników badań, normalizują je do klas HealthDaily / TrainingDaily i zapisują w bazie DuckDB.

2. Agregacja kontekstu historycznego:

HealthRepository wczytuje historię pomiarów. ContextBuilder oraz TrendEngine budują obiekt HealthContext wyliczający baseline i odchylenia (dla HRV, RHR, snu).

3. Wykonywanie analiz analitycznych:
 - RecoveryEngine wylicza punkty regeneracji (Recovery Score 0-100) i status.
 - PerformanceEngine oblicza wskaźniki obciążenia (CTL, ATL, TSB).
 - WorkoutRepository pobiera ostatnio wykonany trening.
4. Budowa jednolitego stanu (AthleteState):
AthleteStateBuilder łączy wszystkie wskaźniki w jednolity obiekt reprezentujący aktualny stan zawodnika.
5. Podejmowanie decyzji (DecisionEngine):
DecisionEngine analizuje AthleteState pod kątem ograniczeń zmęczeniowych i gotowości, po czym wybiera właściwą recepturę treningową (WorkoutPlan / DecisionResult).
6. Generowanie treningu i raportu:
MorningRenderer buduje raport poranny, a PlannerEngine kompiluje gotowy trening.

3. PIPELINE DECYZYJNY

- A. Diagnosis Engine (decision/diagnosis/engine.py):
Ocena kondycji zawodnika na podstawie AthleteState i wyznacza:
 - Readiness (PEAK, HIGH, MODERATE, LOW)
 - Training Capacity (HIGH, MODERATE, LOW)
 - Injury Risk (LOW, MODERATE, HIGH) - np. przy zmęczeniu ≥ 80 ryzyko staje się HIGH.
- B. Prescription Engine (decision/prescription/engine.py):
Przekształca diagnozę zawodnika w cele treningowe TrainingPrescription:
 - Risk HIGH / Readiness LOW -> RECOVERY (45 min, 30 TSS)
 - Readiness PEAK / Cap HIGH -> VO2MAX (60 min, 80 TSS)
 - Readiness HIGH / Cap HIGH -> THRESHOLD (75 min, 75 TSS)
 - Readiness MODERATE -> ENDURANCE (90 min, 60 TSS)
- C. Selection Engine (decision/selection/engine.py):
Mapuje cel treningowy na dyscyplinę, typ treningu (WorkoutType) oraz intensywność (TSS/h), tworząc obiekt WorkoutPlan z klasą DecisionResult.
- D. Generowanie rekomendacji:
Wynik przekazywany jest do widoku porannego (MorningRenderer) oraz do modułu PlannerEngine.

4. GENEROWANIE TRENINGOW

- Architektura Planner (planner/engine.py):
1. Selection: SelectionEngine dobiera najlepszy przepis z katalogu.
 2. Parsing: DSLParser przekształca przepis na drzewo obiektów DSL.
 3. Compilation: DSLCompiler spłaszcza drzewo DSL do postaci wykonalnych bloków PlannedBlock.

Training Recipes (planner/catalog/recipes.py):
Każdy przepis określa Identity (nazwa, DSL key), Stimulus (system energetyczny, obciążenia), Prescription (czas trwania, cel TSS, struktura) oraz Selection Profile (min/max duration).

Domenowy Język Treningowy (DSL) (planner/dsl/):

- Interval(name, duration, power_from, power_to, cadence_from, cadence_to)

- Repeat(count, children)
- Workout(name, children)

Pozwala na tworzenie struktur interwałowych (np. 4x8 min na progu, 5x5 min V02 Max).

Kompilacja treningu (planner/dsl/compiler.py):

DSLCompiler rekurencyjnie przechodzi po drzewie i rozwija pętle Repeat, generując płaską listę bloków PlannedBlock.

5. SYSTEM WYKONANIA TRENINGU

- ExecutionEngine (execution/engine.py):

Makro-analiza poziomu wykonania. Porównuje czas trwania i TSS z planem. Wylicza overall_score i flagę completed (score >= 90%).

- ExecutionAnalyzer (execution/analyzer.py):

Ocenia ogólne wykonanie na podstawie ważonej oceny 4 wskaźników z scoring.py:
 $\text{Execution Score} = 0.50 * \text{Power} + 0.20 * \text{Cadence} + 0.10 * \text{HR} + 0.20 * \text{Completion}$

- WorkoutExecution (execution/workout_execution.py):

Buduje linię czasu treningu (TimelineBuilder) i wykonuje mikro-analizę blok po bloku.

- BlockExecution (execution/block_execution.py):

Struktura danych przechowująca docelowe i zmierzone wartości mocy, kadencji, tętna oraz punktację wykonania konkretnego bloku interwałowego.

Analiza wykonania treningu:

Trening jest dzielony na bloki na osi czasu. Z pliku aktywności wyciągane są fragmenty odpowiadające danemu okienku czasowemu i wyliczany jest uchyb mocy oraz kadencji.

6. TESTY

Struktura katalogu tests/:

```
tests/
├── decision/           (test_decision_engine.py)
├── diagnosis/         (test_diagnosis_engine.py)
├── dsl/               (test_dsl_builders.py)
├── platform/          (test_coach_scenarios.py, test_platform_engine.py)
├── selection/         (test_recipe_scorer.py)
├── test_readiness.py
└── helpers.py
```

Obecne pokrycie testami:

- Dobrze przetestowane: wyliczanie regeneracji, dobór receptur, scenariusze decyzyjne, DSL.
- Łuki: brak testów dla pakietu execution/, repozytoriów danych i parserów plików.

Wyniki uruchomienia (.venv/bin/pytest -v):

24 testy zakończone sukcesem w 0.06s (100% PASSED).

7. OCENA ARCHITEKTURY I REKOMENDACJE

Mocne strony:

1. Ścisłe rozdzielenie logiki eksperckiej od LLM (deterministyczny silnik w Pythonie).
2. Czytelny język DSL i łatwa kompilacja do formatu ZWO / bloków.
3. Wysoka wydajność bazy analitycznej DuckDB.
4. Modułowa architektura domenowa z silnym typowaniem.

Potencjalny dług techniczny:

1. Dublowanie modeli (np. workout/models.py vs planner/models.py).
2. Wartości progowe wkodowane na sztywno w silnikach (hardcoded thresholds).
3. Pliki i testy placeholderowe (test_diagnosis_engine.py).

Obszary dalszego rozwoju:

1. Dopisanie testów dla modułu execution (analiza blokowa aktywności).
2. Przeniesienie progi fizjologiczne do konfigurowalnych parametrów.
3. Ujednolicenie struktur modeli treningowych w całym projekcie.

=====